Assessment Rubric

Cross-Functional Microservices Project Assignment

Module	Software Architecture and Process Models
Student Level	Level 2
Assignment Duration	7 weeks
Document	05 of 07
Total	100 marks
Team Component	95 marks
Individual Contribution	5 marks

1. Mark Allocation

Criterion	Marks	Mark Type
Project initiation, planning, cross-team coordination, and Jira usage	15	Team
Requirements analysis	15	Team
Architecture and technical design	20	Team
Software implementation and integration	25	Team
Testing and quality assurance	15	Team
Deployment, demonstration, and documentation	5	Team
Individual contribution	5	Individual
Total	100	

2. Performance Band Guidance

Band	Indicative Level
Excellent	Complete, coherent, evidence-based work; strong technical decisions; reliable integration; high-quality execution; clear reflection.
Good	Substantially complete and correct; minor gaps; sound technical decisions; working integration; credible evidence.
Satisfactory	Minimum requirements met with noticeable limitations, uneven quality, or incomplete evidence; core prototype works.
Limited	Major gaps, weak justification, partial implementation, unreliable integration, or insufficient evidence.
Inadequate	Little relevant evidence, non-functional or isolated work, serious integrity issues, or failure to meet the required scope.

3. Detailed Rubric

Criterion	Excellent	Good	Satisfactory	Limited	Inadequate
1. Project initiation, planning, cross-team coordination, and Jira usage (15)	13-15: Clear proposal and scope; realistic plan; well-structured Jira epics/stories; acceptance criteria; visible sprint evidence; active risk/change/dependency management; strong weekly cross-team synchronization and progress reporting.	10-12: Good planning and Jira use with minor inconsistencies; risks and dependencies mostly managed; cross-team coordination is regular and evidenced.	7-9: Basic proposal, backlog, and sprint tracking exist; planning or acceptance criteria are uneven; dependencies and changes are handled reactively.	4-6: Weak or outdated plan; Jira is incomplete or used mainly after work; limited risk or cross-team coordination evidence.	0-3: No credible planning process, minimal Jira evidence, unmanaged scope, or no cross-team coordination.
2. Requirements analysis (15)	13-15: Coherent SRS; complete functional/non-functional requirements; high-quality user stories and acceptance criteria; business rules; use cases/user flows; traceability; cross-domain workflows and scope boundaries are clear.	10-12: Requirements are mostly complete and consistent; minor gaps in traceability, rules, or cross-domain detail.	7-9: Core requirements are present but some are ambiguous, unprioritized, weakly tested, or inconsistent with implementation.	4-6: Important requirements or user roles are missing; stories lack acceptance criteria; scope is unclear or frequently changes without control.	0-3: Requirements are absent, copied without adaptation, or do not correspond to the delivered system.
3. Architecture and technical design (20)	17-20: Well-justified microservice boundaries; independent data ownership; API Gateway and JWT design; clear APIs; robust error/security approach; deployment architecture; ERDs; frontend modularity; ADRs; cross-service failure handling; diagrams match implementation.	13-16: Sound architecture with clear services and data ownership; minor weaknesses in contracts, security depth, failure handling, ADRs, or diagram consistency.	9-12: Basic microservice design works but boundaries, data ownership, contracts, security, or deployment design are uneven or weakly justified.	5-8: Architecture is largely conceptual; services share data improperly; diagrams conflict with code; security and integration design are weak.	0-4: Monolithic or disconnected implementation presented as microservices; no credible architecture or major security/design failures.
4. Software implementation and integration (25)	22-25: Secure and maintainable services; meaningful business rules; strong frontend modules; robust validation and errors; clean persistence; reliable cross-team workflows; reviewed pull requests; useful CI; deployed integrated release; no critical defects.	17-21: Core workflows work and integration is credible; minor defects or uneven code quality; good GitHub collaboration and CI evidence.	12-16: Functional prototype demonstrates minimum scope but integration is fragile, features are incomplete, or code quality and collaboration are inconsistent.	6-11: Only partial features work; services are isolated; substantial manual workarounds; weak GitHub practice; major defects affect core workflows.	0-5: Little working software, no meaningful integration, inaccessible repositories, or copied/unverifiable implementation.
5. Testing and quality assurance (15)	13-15: Comprehensive test strategy and cases; meaningful unit, API, integration, system, security, and UAT evidence; defects tracked to closure; test data and results are credible; performance observations are reported.	10-12: Good test coverage and evidence with minor gaps; defects are managed and core workflows are verified.	7-9: Basic unit/API/system testing exists but important edge cases, integration failures, security, or defect evidence are incomplete.	4-6: Testing is mostly manual or superficial; results are poorly recorded; defects remain unexplained; weak traceability to requirements.	0-3: No credible test evidence, fabricated results, or critical failures are ignored.

Criterion	Excellent	Good	Satisfactory	Limited	Inadequate
6. Deployment, demonstration, and documentation (5)	5: Reproducible Docker Compose setup; accessible deployment; concise release notes; clear user/admin/deployment/API documentation; professional presentation and reliable live demo.	4: Deployment and demo work with minor setup or documentation issues.	3: Local or cloud deployment works partially; demonstration needs assistance; documentation has gaps.	2: Significant setup problems, unclear instructions, or unreliable demo.	0-1: No usable deployment or demonstration; documentation is missing or unusable.
7. Individual contribution (5)	5: Sustained, substantial, and traceable contribution across implementation, review, testing, Jira, and/or documentation; high-quality pull requests and collaboration; contribution statement matches evidence.	4: Consistent and meaningful contribution with good evidence; minor imbalance or gaps.	3: Adequate contribution but narrow, intermittent, or weakly documented.	2: Limited contribution relative to role; low-quality or late evidence; little review or integration participation.	0-1: Minimal, unverifiable, inappropriate, or academically dishonest contribution.

4. Individual Contribution Evidence

- GitHub commits with meaningful content, not only formatting or generated files.
- Pull requests authored, reviewed, corrected, and merged.
- Jira issues assigned, updated, and completed with evidence.
- Owned requirements, designs, tests, database work, deployment work, or documentation.
- Participation in integration fixes, release preparation, and team or cross-team coordination.
- A concise individual contribution statement linking evidence to repositories, pull requests, Jira issues, and deliverables.

Moderation rule

The team mark is shared for Criteria 1-6 unless the teaching team has clear evidence of non-participation, misconduct, or a major difference in contribution. Criterion 7 is marked individually. There is no individual viva.

5. Penalty and Integrity Principles

- Unattributed copied code, fabricated Jira/GitHub evidence, falsified tests, or unauthorized collaboration may be handled under academic integrity procedures.
- Direct commits to protected branches, committed secrets, or unsafe handling of real personal data may reduce relevant technical or process marks.
- A feature demonstrated only through mocked screenshots without working code does not count as implemented.
- Late or inaccessible evidence may not be considered unless an approved extension applies.
- Optional advanced technology does not compensate for missing mandatory functionality, integration, security, or documentation.